PhishGuard AI — Academic Project Report (Preliminary)

Status: Source code and test evidence not supplied. Replace placeholders, verify claims, execute tests, and obtain guide approval before submission.

Front matter

Title page

PhishGuard AI

Student: [Name] · USN: [USN] · Degree/Branch: [BE/Branch] · College: [College] · University: [University] · Guide: [Guide] · Academic year: [Year]

Certificate template

This is to certify that [Student, USN] submitted the project titled PhishGuard AI in partial fulfillment of [degree/branch] at [institution] during [year]. Guide: [name/signature] · HOD: [name/signature] · Date/place: [fill]. **Template only; no approval or completion is asserted.**

Declaration template

I, [name], declare this report is my work under [guide], and that sources are acknowledged. All implementation/test statements will be checked against artifacts. Signature/date: [fill].

Acknowledgement

[Write only genuine acknowledgements; do not invent names.]

Abstract

PhishGuard AI is described as a browser-based demonstration for email and URL phishing-warning analysis. The brief states: user-provided email text/URLs and synthetic scenarios; local logic: local heuristics, URL parsing, and explainable risk scoring. Its purpose is to demonstrate explainable local analysis, not production-grade security. Source code, screenshots, and test results were not provided, so implementation and outcomes remain unverified. Future work includes external AI, reputation/threat intelligence, SPF/DKIM/DMARC checks, malware scanning, and live-site analysis.

Table of contents

1 Introduction · 2 Literature Review · 3 Requirements Analysis · 4 System Design · 5 Methodology and Implementation · 6 Testing · 7 Results and Discussion · 8 Security/Ethics/Limitations · 9 Conclusion · References · Appendices

List of figures/tables

[Populate after inserting genuine diagrams/screenshots and final tables.]

Abbreviations

AI—Artificial Intelligence; API—Application Programming Interface; CSS—Cascading Style Sheets; HTML—HyperText Markup Language; SIEM—Security Information and Event Management; SRS—Software Requirements Specification; UI—User Interface; URL—Uniform Resource Locator; XSS—Cross-Site Scripting.

Chapter 1: Introduction

Background

Cybersecurity tools help organize signals, but classifications can be wrong. An educational prototype should expose inputs, rules, explanations, and limitations.

Problem statement

Users may need help interpreting email and URL phishing-warning analysis. This project proposes a controlled local workflow; it does not establish operational security coverage.

Motivation/objectives

Make the concepts approachable through a browser demo; demonstrate local analysis; show explanations; label synthetic/simulated behavior; define tests and roadmap.

Scope/users/contribution

Scope: browser-based demonstration and synthetic or authorized inputs. Users: student operator and evaluator. Contribution: low-friction, explainable learning workflow. Exclude production deployment and external capabilities unless verified.

Chapter 2: Literature Review and Existing Systems

Concepts

Phishing/social engineering, URL structure, email indicators, user awareness, and heuristic classification are relevant. Rule systems are interpretable but depend on rule coverage. Browser-only tools are accessible but lack enterprise telemetry, central management, and operational controls.

Existing approaches/limitations

Manual review, rule-based systems, reputation services, SIEM platforms, and AI classifiers address different use cases. Manual review takes effort; rules can miss novel patterns or flag benign cases; external services require connectivity and privacy review; enterprise platforms need reliable ingestion, authorization, audit, and operations.

Comparison/design gap

This student prototype prioritizes local, explainable demonstration rather than replacing existing tools. The educational design rationale is not a claim of research novelty or measured superiority.

Preliminary sources

Candidate official sources listed below; verify and cite only sources actually consulted.

Chapter 3: Requirements Analysis

Functional requirements are in the companion SRS: Email input/analysis, URL extraction, URL structure analysis, Suspicious-pattern detection, Explainable risk scoring, Evidence display, Awareness training, Analysis history, Report export, Input validation, Privacy controls. All are described in the brief but not source-verified. Non-functional goals include usability, portability, reliability, maintainability, accessibility, privacy, secure input handling, and data integrity; testing is pending.

Environment and feasibility

Development tools reported: VS Code and Claude AI. Runtime described: HTML/CSS/vanilla JavaScript in a modern browser. Exact OS/browser/version: [fill]. Technical feasibility is plausible for a local demo; operational feasibility depends on clear setup; costs and schedule are not evidenced. Assumptions: inputs authorized, fixtures synthetic, source description accurate.

Use cases

Open app/read scope → load or enter input → validate → run local analysis → review evidence/score → optionally export or clear data if supported.

Chapter 4: System Design

Conceptual architecture inferred from brief; revise after source inspection. Future integrations are not included as implemented.

Architecture/data flow

```
flowchart LR
U[User] --> UI[Browser UI: HTML/CSS]
UI --> JS[Vanilla JavaScript]
```

```
I[Synthetic or authorized input] --> JS
JS --> R[Local rules / parsing / scoring]
R --> O[Finding and explanation]
O --> UI
JS <--> L[Browser storage, if confirmed]
UI --> E[Export, if confirmed]
```

Component/use-case/activity/sequence diagrams

```
flowchart TB
UI[Presentation] --> V[Input validation]
V --> A[Local analysis]
A --> X[Risk/explanation formatter]
X --> UI
A --> H[Local history, if present]
```

```
flowchart LR
User((User)) --> I[Provide input]
User --> R[Run analysis]
R --> E[Review explanation]
E --> X[Export/clear if supported]
```

```
flowchart TD
S([Start]) --> I[Enter/load input]
I --> V{Valid?}
V -- No --> M[Show error] --> I
V -- Yes --> A[Run local logic] --> O[Show findings/limits] --> Q{Continue?}
Q -- Yes --> I
Q -- No --> E([End])
```

```
sequenceDiagram
actor User
participant UI as Browser UI
participant Logic as Local JS
User->>UI: Submit input
UI->>Logic: Validate/analyze
Logic-->>UI: Findings/explanation
UI-->>User: Display result and caveat
```

Data model/modules/trust boundaries

Exact schemas are unknown. Candidate fields: local ID, time, input type, finding, explanation, score, synthetic marker. Minimize stored raw content. Trust boundaries: user input→logic/UI; app↔browser storage; browser→download. Future external services require auth, authorization, transport security, and data minimization.

Chapter 5: Methodology and Implementation

Method

Iterative process: requirements, browser module, synthetic test input, observe output, test normal/negative cases, revise. Actual dates/commits: [insert].

Technology

HTML structure, CSS presentation, vanilla JavaScript logic, browser APIs only as confirmed. Source filenames/API calls: [map from repository].

Modules

Module	Purpose/inputs	Processing/output	Error/security notes	Status
Input/fixtures	user-provided email text/URLs and synthetic scenarios	Validate/load	Synthetic labels; length bounds	Described; verify
Analysis	Validated input	local heuristics, URL parsing, and explainable risk scoring	Explain rules; malformed input handling	Described; verify
Score	Findings	Formula/weights [source needed]	Not a probability unless calibrated	Unverified
UI	Analysis result	Show finding/evidence	Safe rendering and caveats	Unverified
History/export	Result/history	Browser storage/download if present	Privacy, deletion, data minimization	Unverified

Module	Purpose/inputs	Processing/output	Error/security notes	Status
Workflow	User selection/scenario	awareness scenario	Avoid claims of live operation	Described; verify

Implementation detail

Exact rules, thresholds, score weights, storage keys, file names, and source excerpts must be added from actual source. Do not insert pseudocode as code that was implemented.

Chapter 6: Testing and Validation

Objectives: test functional paths, boundaries, malformed inputs, persistence/export, privacy, accessibility, and safe wording. Environment/date/build: [record]. Use companion test plan and traceability matrix. No tests were supplied; **all actual results are Not yet executed**. Planned unit checks cover rule/parser logic; integration checks cover UI↔logic↔storage/export; negative checks cover empty/large/malformed inputs; privacy checks inspect storage/network. Log defects and retest after fixes.

Chapter 7: Results and Discussion

Screenshots

- Figure 7.1 [Insert genuine application home/dashboard screenshot]
- Figure 7.2 [Insert genuine analysis/result screen]
- Figure 7.3 [Insert genuine history/export/workflow screen if implemented]

Do not use mockups as actual evidence.

Findings

Fill from executed test logs only. The brief describes workflows but does not prove they ran. A test result establishes behavior only for the tested inputs/build/environment; it does not prove general detection accuracy or production readiness.

Evaluation template

Dataset/version	Cases	TP	FP	TN	FN	Precision	Recall	Notes
[fill after running]								

Chapter 8: Security, Ethics, and Limitations

Use authorized/synthetic data; never include credentials or private content in examples. Treat input/storage as untrusted and render safely. Browser-local storage is not encrypted or tamper-proof. Rule-based analysis can produce false positives and false negatives. URL heuristics do not verify reputation or live behavior and cannot guarantee phishing detection.. Future services introduce privacy, credentials, availability, and integration risks.

Chapter 9: Conclusion and Future Scope

The project is described as an explainable browser-based demonstration. Achievement against objectives remains pending source review and tests. Current limitations include local logic, uncertain rule coverage, and no verified production controls. Future scope: external AI, reputation/threat intelligence, SPF/DKIM/DMARC checks, malware scanning, and live-site analysis. Add integrations only with authorization, privacy review, robust security design, and validation.

References (candidate list; consult and verify before citing)

1 MITRE, “MITRE ATT&CK,” https://attack.mitre.org/ (CyberShield background; consult relevant pages).

2 Wazuh, “Wazuh Documentation,” https://documentation.wazuh.com/ (roadmap only, not proof of integration).

3 OWASP, “Cross Site Scripting Prevention Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html.

4 OWASP, “Phishing,” https://owasp.org/www-community/attacks/Phishing.

5 MDN Web Docs, “Window.localStorage,” https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage.

6 MDN Web Docs, “URL,” https://developer.mozilla.org/en-US/docs/Web/API/URL.

Use numbered in-text citations only for sources actually read; add access dates after consultation.

Appendices

User guide

Open the project folder; identify entry HTML from README/source; open in supported browser using tested method; read scope; use synthetic/authorized input; run workflow; inspect explanation; export/reset only if confirmed. Exact filenames/screenshots: [fill].

Installation-free setup

No core installation is stated. Test opening entry HTML and whether browser restrictions require a local server; document observed steps only.

Test checklist

[ ] source inventory [ ] browser/build recorded [ ] tests executed [ ] negative inputs [ ] privacy/network [ ] real screenshots [ ] defects retested [ ] matrix updated

Synthetic data

[Describe fixture origin, fields, generation, labels, and synthetic nature from source.]

Report sample

[Insert a genuine app-generated export, or leave blank until one is generated.]

Troubleshooting

Page load: check entry file/browser/console. Analysis: check required inputs and console. History: inspect origin/storage permissions. Export: inspect browser downloads and actual format. Unexpected result: preserve input and rule version.

Demo procedure

Explain scope; show synthetic input; run analysis; explain score/evidence without overclaiming; show verified history/export; state limitation and roadmap; avoid private data.

Glossary

False positive—benign item flagged. False negative—threat missed. Heuristic—approximate rule. Synthetic fixture—fabricated test record. Trust boundary—component/data boundary.